Network Instrusion Detection Using Classical AI and Machine Learning

Authors: Qazi Abdurrahman (24p-0570) & Meher Ali (24p-0500)

Abstract

This project develops a simple Network Intrusion Detection System (NIDS) to classify network traffic as either normal or an attack. Utilizing a dataset of 6,000 connection records, we implemented and evaluated multiple AI techniques ranging from a rule-based Simple Reflex Agent to Supervised Classifiers (KNN, Naïve Bayes, Logistic Regression) and Unsupervised Clustering (K-Means). Finally, a Genetic Algorithm was used for feature selection. Overall, the K-Nearest Neighbors (k=1) classifier achieved the best performance with an F1score of 0.9579.

Introduction

Network administrators are tasked with monitoring thousands of connection records every minute. Inspecting every record manually to detect malicious activity is practically impossible. Therefore, automated Network Intrusion Detection Systems (NIDS) are critical for modern cybersecurity, as they can rapidly inspect connections and predict whether they constitute an attack.

In this project, we build a simple NIDS using various Artificial Intelligence and Machine Learning techniques. We begin by establishing a baseline using a hand-crafted Simple Reflex Agent. We then progress to training classical supervised models, exploring unsupervised patterns using K-Means clustering, and optimizing our feature space using a Genetic Algorithm. This progression demonstrates the shift from rigid, rule-based logic to adaptive, data-driven learning.

Dataset

The dataset used is `network_traffic.csv`, a simplified derivative of the NSL-KDD dataset. It contains balanced records with all features being purely numeric, meaning no one-hot encoding or label encoding was required.

- **Rows:** 6,000 (balanced: 3000 normal + 3000 attack)
- **Features:** 15 numeric features detailing connection metrics (e.g., duration, bytes, error rates).
  - **Target:** `label` (0 = normal, 1 = attack)

Task 1: Data exploitation

Initial data exploration revealed a perfectly balanced class distribution with 3,000 instances per class. We plotted feature histograms for variables such as `src_bytes` and `dst_bytes`. The statistical summary indicated drastic scale differences between features; for instance, `src_bytes` had a maximum value in the millions, whereas `serror_rate` fell between 0 and 1. This underscored the necessity of applying `StandardScaler` for distance-based algorithms like KNN.

Task 2: Simple reflex Agent

A simple reflex agent operates solely on the current percept using static if-then rules, possessing no internal state or learning capability. We designed our baseline agent by setting thresholds on suspicious indicators. The agent flags a connection as an attack (1) if: `wrong_fragment` > 0, or `hot` >= 5, or `num_compromised` > 0, or if the user is not logged in (`logged_in` = 0) while `serror_rate` >= 0.5.

This rule-based approach yielded an accuracy of 0.8575 and an F1-score of 0.8379. While highly interpretable and precise (Precision = 0.9714), its recall was low (0.7367), meaning static rules fail to capture complex, overlapping feature interactions and miss many attacks.

Task 3: Supervised algorithem

We trained three classical models using an 80/20 train-test split. To account for varying feature scales, `StandardScaler` was applied to Logistic Regression and K-Nearest Neighbors (KNN). We utilized crossvalidation to tune KNN, finding k=1 to be the optimal hyperparameter. Naïve Bayes was trained without scaling. The results are summarized in Table 1.

Model	Accuracy	Precision	Recall	F1
Simple Reflex Agent	0.8575	0.9714	0.7367	0.8379
KNN (best k=1)	0.9575	0.9493	0.9667	0.9579
Naïve Bayes	0.7875	0.9367	0.6167	0.7437
Logistic Regression	0.9150	0.9249	0.9033	0.9140

Comparison of classifiers on the test set.

Task 4: Clustering with K-means

To test an unsupervised approach, we removed the labels and applied K-Means (k=2) to the scaled feature set. Because cluster IDs are arbitrary, we mapped each cluster to a class label using a majority vote from the training set. The mapped K-Means approach achieved an accuracy of 0.8533 and an F1-score of 0.8394. While it successfully grouped normal vs. attack traffic, it predictably lagged behind supervised models since it had no access to ground-truth labels during training.

Task 5: genetic algorithem for feature selection

We designed a binary Genetic Algorithm (GA) to perform feature selection. Each chromosome was a 15-bit vector where '1' meant the feature was included. Fitness was evaluated using the validation F1-score of a Logistic Regression model. Implementing tournament selection, crossover, and mutation over 25 generations, the GA successfully reduced the feature space from 15 down to 11 features. The reduced model achieved a validation F1-score of 0.9130, proving that we can discard unnecessary features to create a faster, simpler model without sacrificing predictive power.

Discussion

KNN (k=1) achieved the best overall performance, demonstrating that network attacks in this dataset form tight, localized clusters in the scaled feature space. Logistic Regression also performed strongly, proving that linear decision boundaries are highly effective here. The Simple Reflex Agent struggled primarily with recall, heavily illustrating the limitations of hard-coded thresholds in cybersecurity. Feature selection via GA was highly beneficial, confirming that removing noisy or redundant features simplifies the model with negligible loss in accuracy. If given more time, we would explore ensemble methods like Random Forests or Gradient Boosting to see if we could push the accuracy beyond 95%.

Conclusion

This project successfully demonstrates the evolution of an AI intrusion detection system. We proved that while simple reflex rules provide a decent baseline, supervised machine learning—specifically KNN and Logistic Regression—vastly improves threat detection capabilities. Furthermore, optimizing the system with a Genetic Algorithm ensures efficiency, making the system robust and suitable for real-world network traffic monitoring.

Statement of contribution

Qazi Abdurrahman (24p-0570): Handled the data exploration (Task 1), implemented the K-Means clustering algorithm alongside the PCA visualizations (Task 4)

Meher Ali (24p-0500): Designed the Simple Reflex Agent rules (Task 2), implemented and tuned the Supervised Classifiers (Task 3), and wrote the core logic for the Genetic Algorithm feature selection (Task 5). And structured the final documentation.